

BASIC CHIP DESIGN FLOW

(OPEN-SOURCE TOOLS BASED)

INTRODUCTION TO CHIP DESIGN FLOW:

Chip design is a systematic and well-defined process that converts a **functional specification** into a **manufacturable silicon chip**. This flow starts from a high-level behavioural description of the design, known as **Register Transfer Level (RTL)**, and progresses through multiple verification, synthesis, and physical implementation stages to finally produce a **GDSII layout**, which is sent for fabrication. The flow moves from high-level design description (RTL) to physical implementation (GDSII) while ensuring functional correctness, timing closure, and manufacturability.

Throughout this flow, special emphasis is placed on:

- **Functional correctness**, ensuring the design behaves as intended.
- **Timing closure**, ensuring the design operates correctly at the desired clock frequency.
- **Manufacturability**, ensuring the layout complies with foundry design rules to achieve a manufacturable silicon layout at the end.

In this document, a **simple end-to-end chip design flow** is explained using **mainly open-source EDA tools**, making it suitable for **academic laboratories, training programs, and beginner-level VLSI learning**. You will find a simplest end-to-end chip design flow using mainly open-source tools, suitable for learning and academic labs.

STEP 1. DESIGN SPECIFICATION

Purpose:

This step defines what the chip should do and under what constraints it should operate. It acts as the foundation of the entire chip design process. Any ambiguity at this stage can lead to functional or performance issues later in the flow.

Input for this process:

- **Product requirements:** Overall functionality expected from the chip
- **Performance targets:** Clock frequency, power consumption, and area expectations
- **Interface description:** Input/output ports, communication protocols, and data widths

Output:

- **Design specification document:** A detailed written description of functionality and constraints
- **Block-level architecture:** High-level block diagram showing major functional blocks and their interconnections

Tools Used:

- No EDA tool
(This step is majorly performed through manual documentation, discussions, and architectural planning.)

STEP 2. RTL DESCRIPTION (HDL)

Purpose: This step describes the **functional behaviour** of the chip using Hardware Description Language (HDL). For writing RTL(Register transfer language) a designer may use languages like Verilog or System Verilog. RTL describes how data flows between registers and how logic operations are performed on each clock cycle.

** NOTE: RTL is technology-independent and focuses purely on **functionality**, not physical implementation.*

Input for this process:

- Design specification document
- Interface definitions

Output Files

These files represent the complete functional description of the design.

- RTL source files Eg: design.v/ design.sv

Tools Used (Open Source)

- Icarus Verilog
- Verilator

STEP 3. FUNCTIONAL VERIFICATION (RTL SIMULATION)

Purpose: In this step a design verification professional verifies **the correctness of the RTL design** before it is handed over for synthesis. The objective is to ensure that the RTL behaves exactly as defined in the specification.

If bugs are detected at this stage, they must be fixed in RTL. **Only bug-free RTL is allowed to move forward to synthesis**, as fixing bugs later in the flow is costly and time-consuming.

Input Files for this process:

- RTL files (*.v, *.sv)
- Testbench files (*.tb.v) that generate stimulus

Output Files:

- Simulation logs showing pass/fail information
- Waveform files (*.vcd) used for debugging and analysis

Tools Used(open source)

- Icarus Verilog
- Verilator
- GTKWave – for viewing and analysing waveforms

STEP 4. LOGIC SYNTHESIS

Purpose: In this step, the verified RTL code is **converted into a gate-level netlist** using standard logic cells. The synthesis process optimizes the design to meet timing, area, and power constraints.

Synthesis bridges the gap between **functional design** and **physical implementation**.

Input Files

- RTL files
- Timing constraints (*.sdc)
- Standard cell library (*.lib)

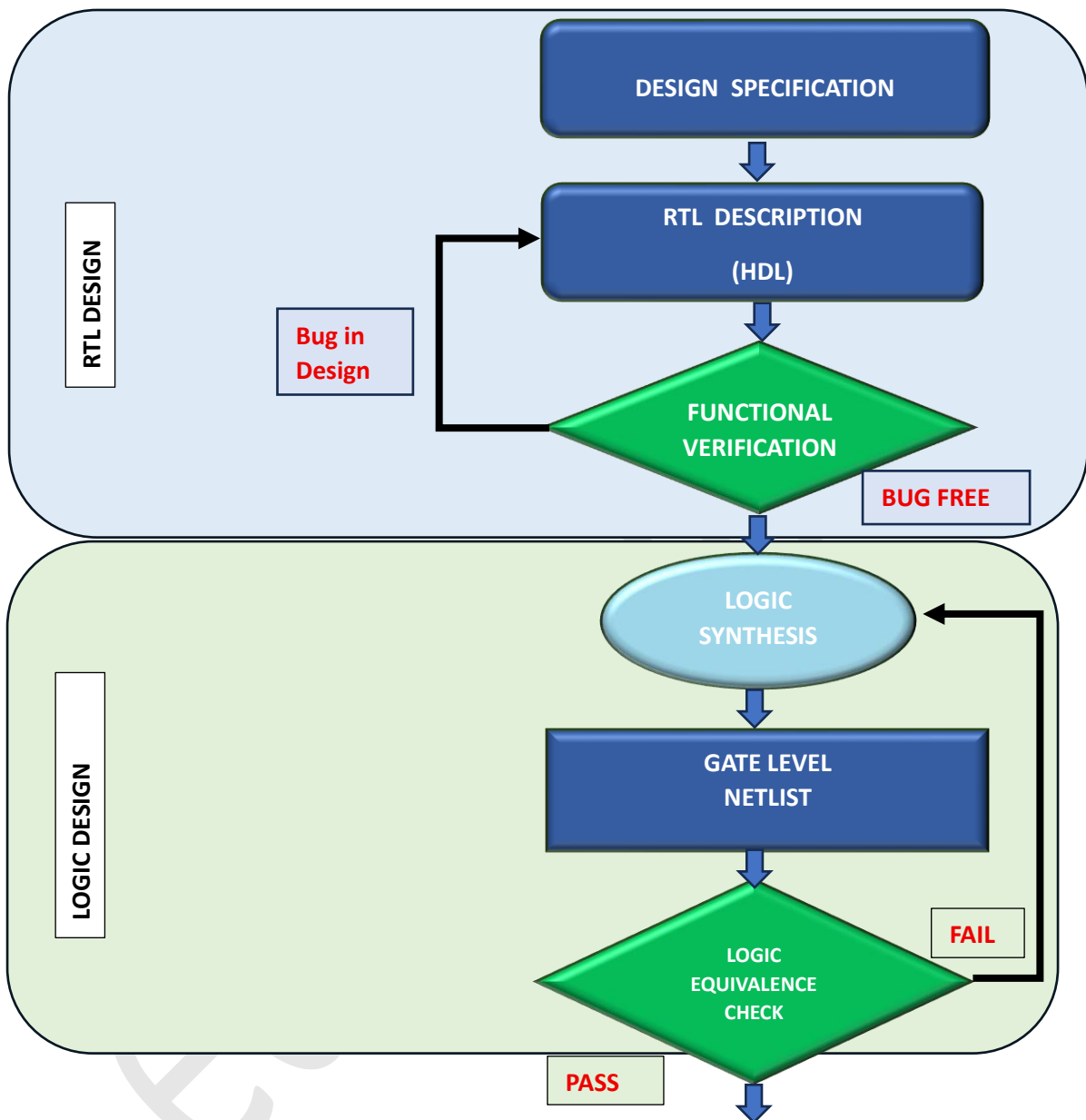
Output Files

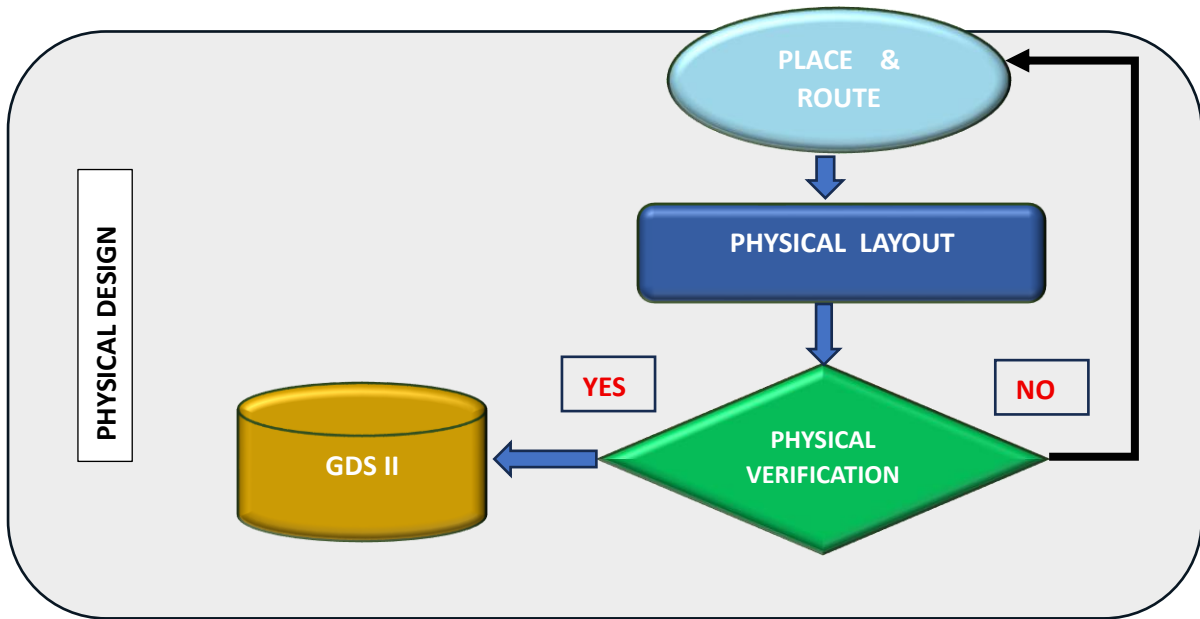
- Gate-level netlist (netlist.v)
- Reports including:
 1. Area report
 2. Timing report
 3. Power estimation report.

Tools Used

- Yosys – open-source synthesis tool
- ABC - logic optimization engine (used internally by Yosys)

(The BUG FREE / PASS / FAIL flow diagram visually represents the iterative nature of RTL verification, synthesis, and equivalence checking, ensuring only correct designs move forward.)





STEP 5. LOGIC EQUIVALENCE CHECK (optional but recommended)

Purpose: This step ensures that the **synthesized gate-level netlist is functionally equivalent to the original RTL design**. It guarantees that synthesis has not altered the intended functionality.

Input Files

- RTL netlist
- Gate-level netlist

Output Files

- Equivalence pass/fail report

Tools Used(open source)

- Yosys (formal equivalence checking)

STEP 6. PHYSICAL DESIGN (PLACE & ROUTE)

Purpose: This step converts the **logical gate-level netlist into a physical layout** that can be manufactured. Physical design determines where each cell is placed on silicon and how they are interconnected.

This stage includes the following sub-processes:

- **Floorplanning:** Defines chip and core area
- **Placement:** Positions standard cells
- **Clock Tree Synthesis (CTS):** Builds a balanced clock network

- **Routing:** Connects all cells using metal layers

Input Files

- Gate-level netlist
- Technology LEF(Library Exchange Format) files (*.lef)
- Timing constraints (*.sdc- synopsys design constraint)
- Liberty files (*.lib)

Output Files

- Placed DEF (*.def) files which describes the physical implementation of a chip. This contains: Cell placement information, Routing information, Pin locations, Die and core area which is used in: Place & Route (Physical Design),
- Routed DEF (*.def)files,
- Timing and congestion reports

Tools Used(open source)

- OpenROAD
- OpenLane

STEP 7. PHYSICAL VERIFICATION

Purpose: This step ensures that the physical layout meets manufacturing rules and matches the logical design. It is a critical step before tape-out.

Checks Performed

- DRC (Design Rule Check) : Ensures layout follows foundry rules
- LVS (Layout vs Schematic) : Ensures layout connectivity matches the netlist

Input Files

- Routed layout(*.def)files
- Netlist
- Technology rule files

Output Files

- DRC report
- LVS report

Tools Used

- Magic VLSI for DRC
- Netgen – for LVS

STEP 8. GDSII GENERATION (FINAL LAYOUT)

Purpose: This step generates the **final GDSII file**, which represents the complete physical layout of the chip. This file is sent to the semiconductor foundry for fabrication.

Input Files:

- Verified physical layout
- Technology files

Output Files:

- GDSII file (*.gds)

Tools Used:

- Magic VLSI
- OpenRoad